

EE 446: TinyML

Lab 5: Keyword Spotting Using TinyML

Spring 2026

Instructor: Dinuka Sahabandu (sdinuka@uw.edu)

Assistant Teaching Professor

Department of Electrical and Computer Engineering

University of Washington



**ELECTRICAL & COMPUTER
ENGINEERING**

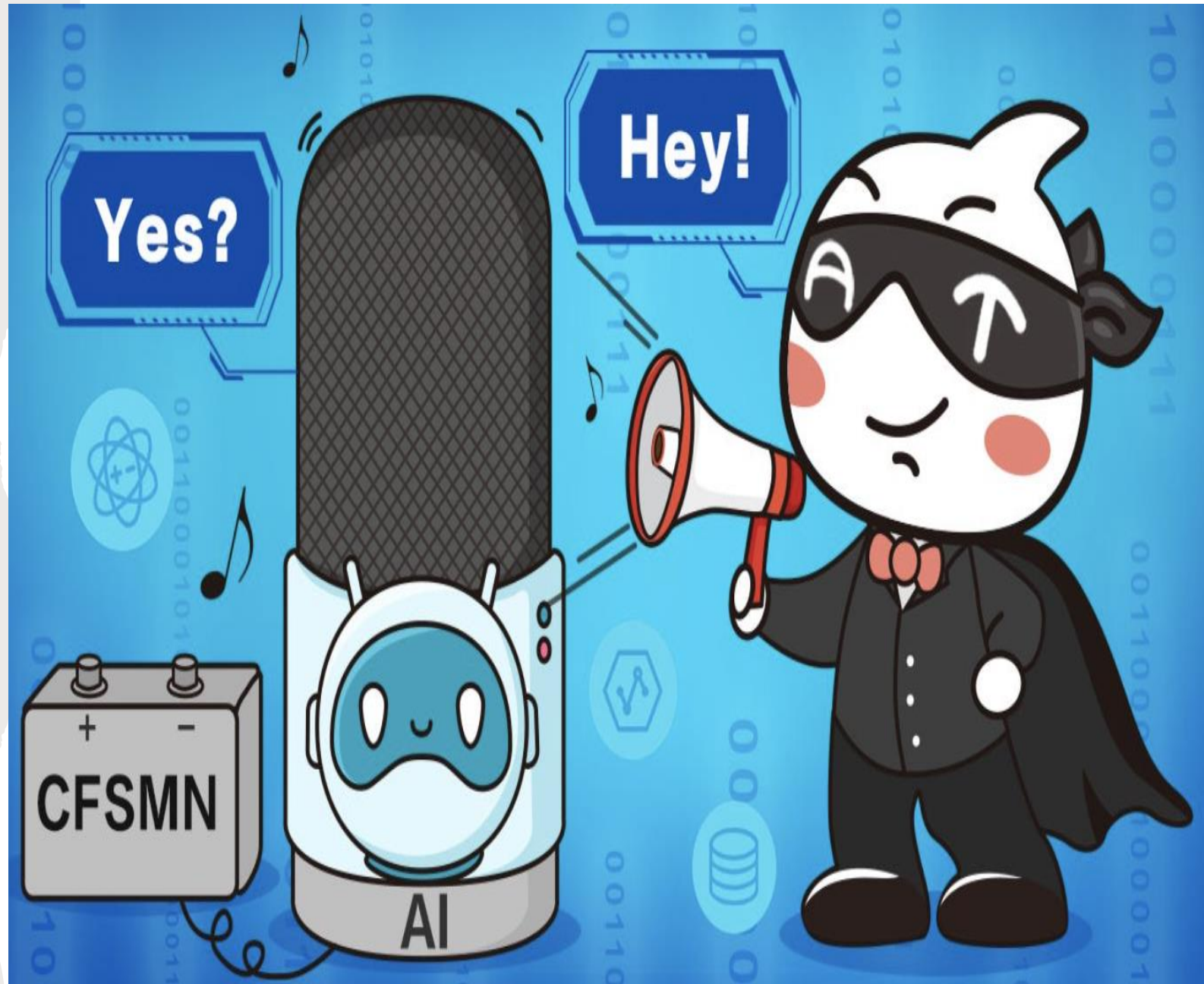
UNIVERSITY of WASHINGTON

30-Apr-26, TinyML, Dinuka Sahabandu



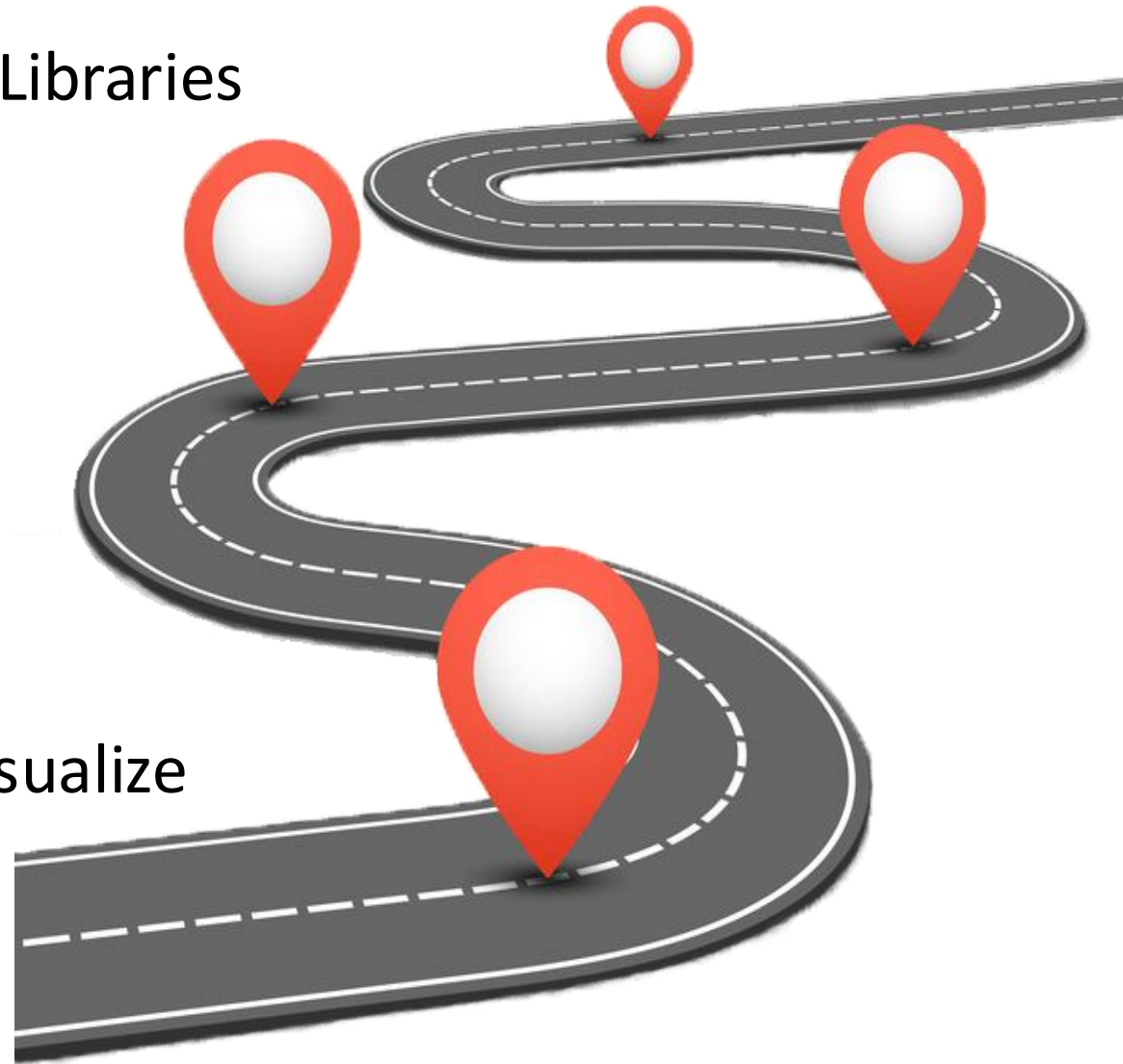
Lab #5

- Both Software and Hardware Lab!
- Loading, Inspecting, and Visualizing Audio Data
- Computing FFT, Spectrogram, and MFCC of Audio Samples
- ML Models for KWS
- Implement the Model with TinyML Kits



Lab 5 – Software - Road Map

1. Installing and Configuring Needed Python Libraries
2. Define Python Audio Importing Function
3. Load in the Audio Samples
4. Visualize the Audio Samples as Signals
5. Compute the FFT of Audio Samples and Visualize
6. Convert FFT to Spectrogram and Visualize



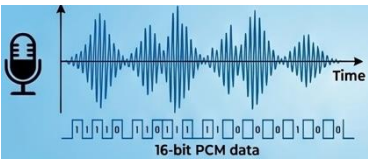
Lab 5 – Software - Road Map

7. Convert Spectrogram to MFCC and and Visualize
8. Import and Configure Tensorflow Libraries
9. Building/Loading ML model for KWS
10. Training ML model for KWS
11. Evaluation of ML Model Performance
12. Create a TensorFlow Lite TinyML model



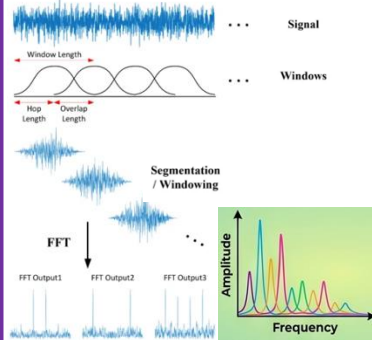
Lab 5 – Software Stack: ML and TinyML Pipeline

Step 1: Raw Audio



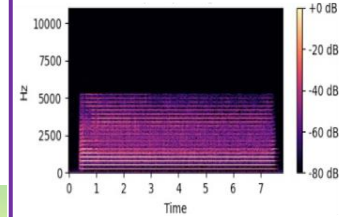
- X = Time (seconds)
- Y = Amplitude (Sound Wave Strength/Volume)
- The raw, digitized sound wave exactly as captured by the microphone.
- *Time data hides specific pitches. We must extract frequency components to identify distinct sounds.*

Step 2: FFT



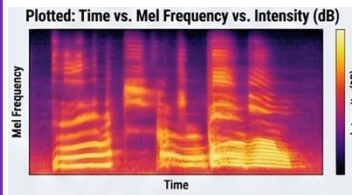
- X = Frequency (Pitch)
- Y = Amplitude (Strength of Pitch)
- A mathematical algorithm that reveals the exact recipe of frequencies that make up the sound.
- *Standard FFT loses the 'time' element. We need a sliding window to see how pitches change as a word is spoken.*

Step 3: Spectrogram



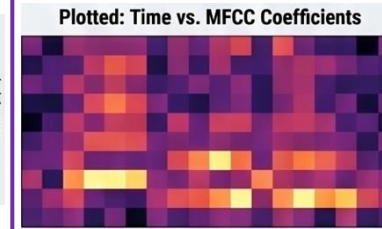
- X = Time
- Y = Frequency (Pitch)
- Color = dB (Energy Intensity/Loudness)
- A visual heatmap showing how the volume of different pitches evolves over time.
- *Linear frequency spacing doesn't match human hearing. We must group pitches the way our ears actually perceive them.*

Step 4: Mel-Spectrogram



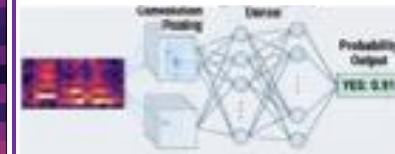
- X = Time
- Y = Mel-Frequency (Human-Perceived Pitch)
- Color = dB
- Stretches low frequencies (where humans hear well) and squashes high frequencies (where we hear poorly).
- *This image is still too noisy and correlated. We must compress it to run on tiny edge-device memory.*

Step 5: MFCC



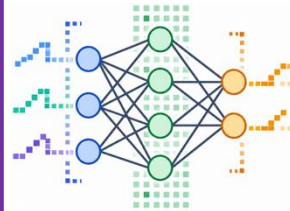
- X = Time
- Y = 13 Coefficients (Acoustic Signature)
- Pixel Color: MFCC magnitude representing the intensity of a specific spectral shape over time.
- A highly compressed, mathematical "fingerprint" defining the unique shape of the spoken word.
- *This compact, decorrelated fingerprint is the ideal lightweight input for a neural network.*

Step 6: CNN



- MFCC Images (inputs) → Convolutions (extract features) → Pooling (reduce size) → Softmax Outputs
- The network recognizes the visual fingerprint and outputs the probability of labels: "Yes", "No", or "Silence".

Step 7: Int8 Quantization



- Converts the trained KWS CNN into an **8-bit integer TFLite model**
- Enables faster, lower-power inference on the Arduino Nano 33 BLE
- Final quantized model is exported as a C/C++ byte array for TFLite Micro

Lab 5 – Software Workflow Options

You may complete the **software part of Lab 5** using either your local class Python environment or Google Colab.

Option 1: Local Python Environment — Recommended

Use the tinym1-arduino Python environment that you created in **Lab 3**. For this option, use:

- TinyML_Lab5_Local_Env.ipynb
- You can [skip the Google Colab setup/upload slides \(Slides 7 & 8\)](#) and open the notebook directly in JupyterLab.
- In Terminal, run:

```
cd ~/ai/projects/tinym1-arduino
source bin/activate
jupyter lab
```

Alternatively, if your setup created the launcher script, you can run:

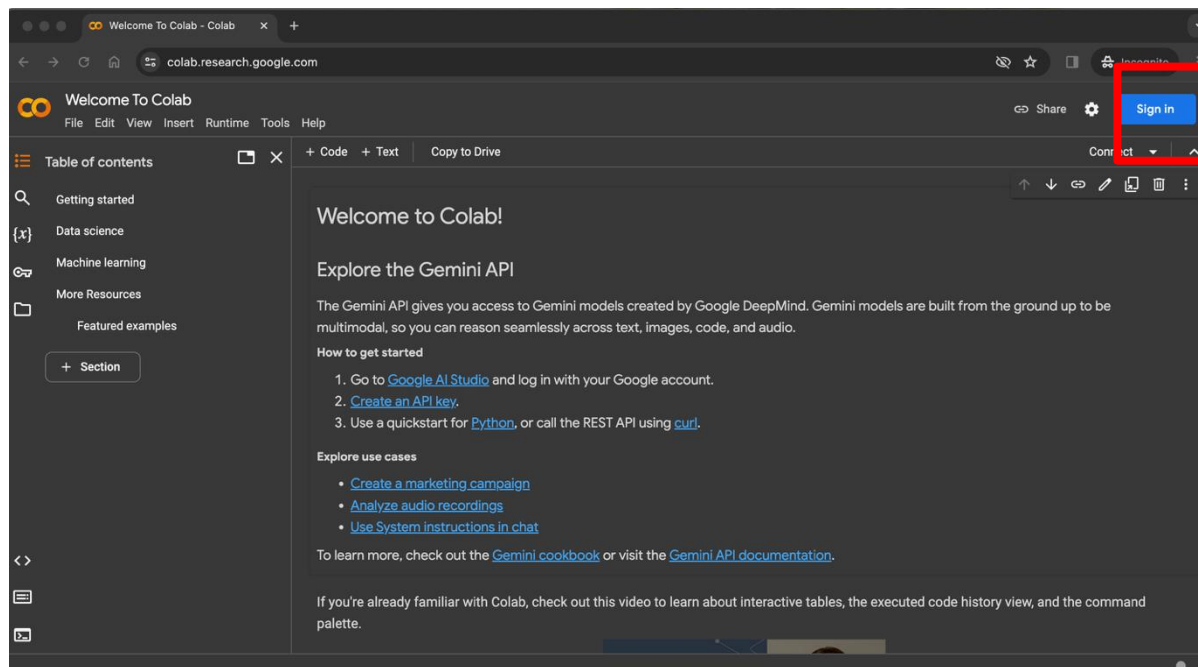
```
~/ai/projects/tinym1-arduino/launch_jupyter.sh
```

Then, in JupyterLab, open `TinyML_Lab5_Local_Env.ipynb` and complete the software part.

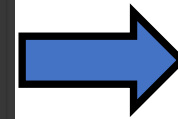
Lab 5 – Software Workflow Options

Option 2: If you are using Google Colab for the Software part of this Lab.

1. Go to website: colab.research.google.com
2. Sign in using your university email account



Step I: Click Here




Sign in
Use your Google Account

Step II: Enter your university email account here

[Forgot email?](#)

Not your computer? Use Guest mode to sign in privately.
[Learn more about using Guest mode](#)

[Create account](#)

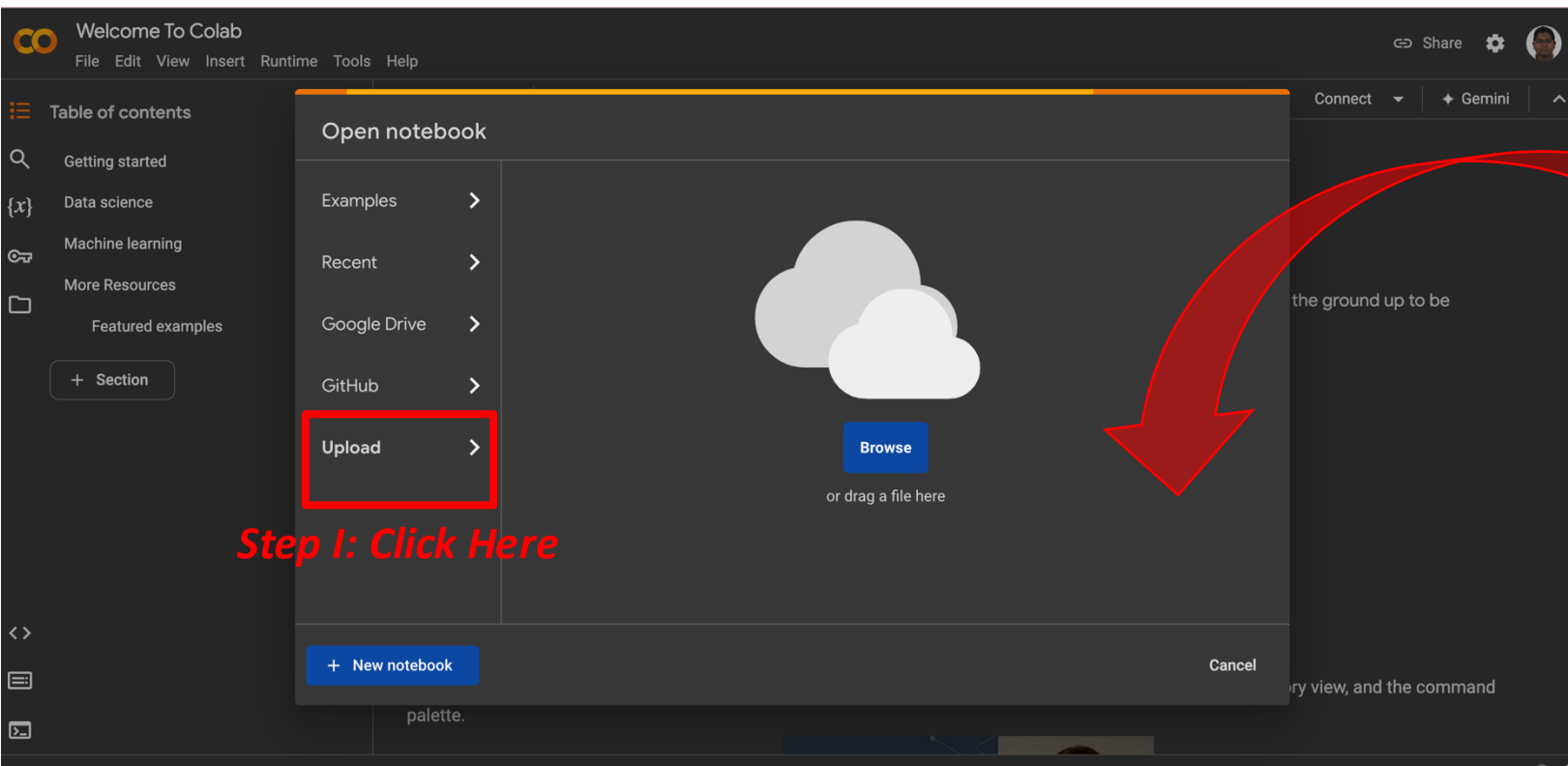


Step III: Click “Next”, enter the account password and login



Lab 5 – Software

3. Upload the provided notebook file *TinyML_Lab5_Google_Colab.ipynb* to Google Colab and follow the instructions in the notebook file.



Step I: Click Here

*Step II: Drag and drop the
Provided notebook (.ipynb) file*

 TinyML_Lab5_Google_Colab.ipynb

Lab 5 - Hardware - Road Map

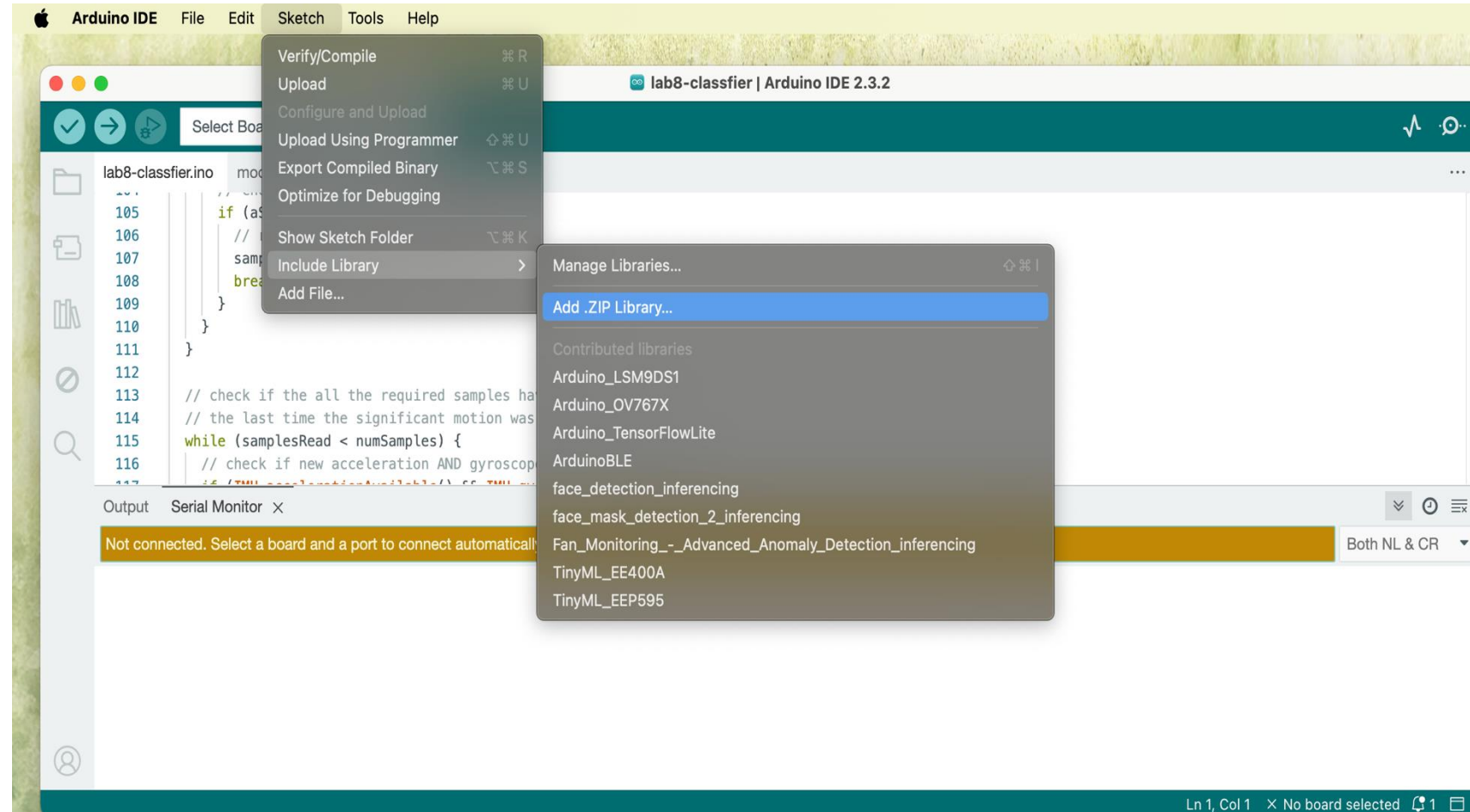
1. Setup needed Libraries in Arduino IDE
2. Open the Arduino Sketch
3. Upload the TinyML model to boards



Lab 5 – Hardware

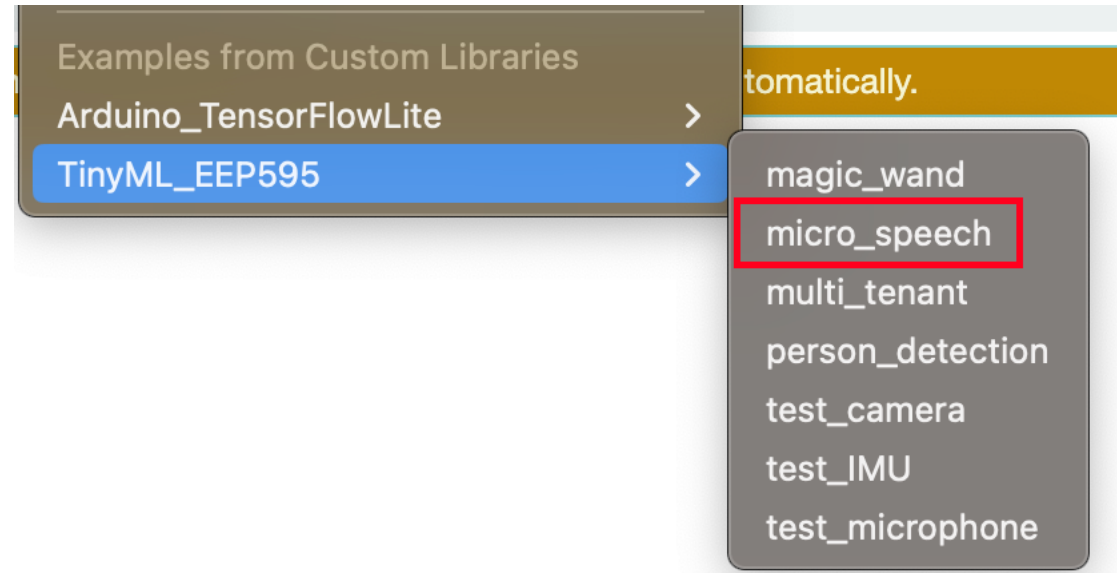
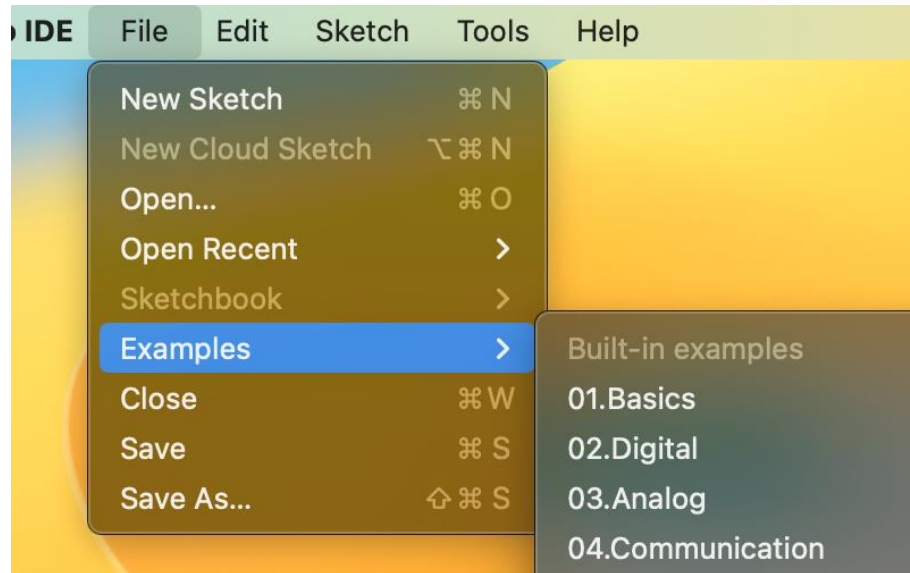
1. Open **Arduino IDE** and add the provided **.zip** files (**Arduino_TensorFlowLite.zip** and **TinyML_EE446.zip**) as Libraries

Note: Make sure to **replace both libraries with the .zip** files provided in this lab, even if they are already linked in your Arduino IDE. **Failure to do so may result in compilation errors.**



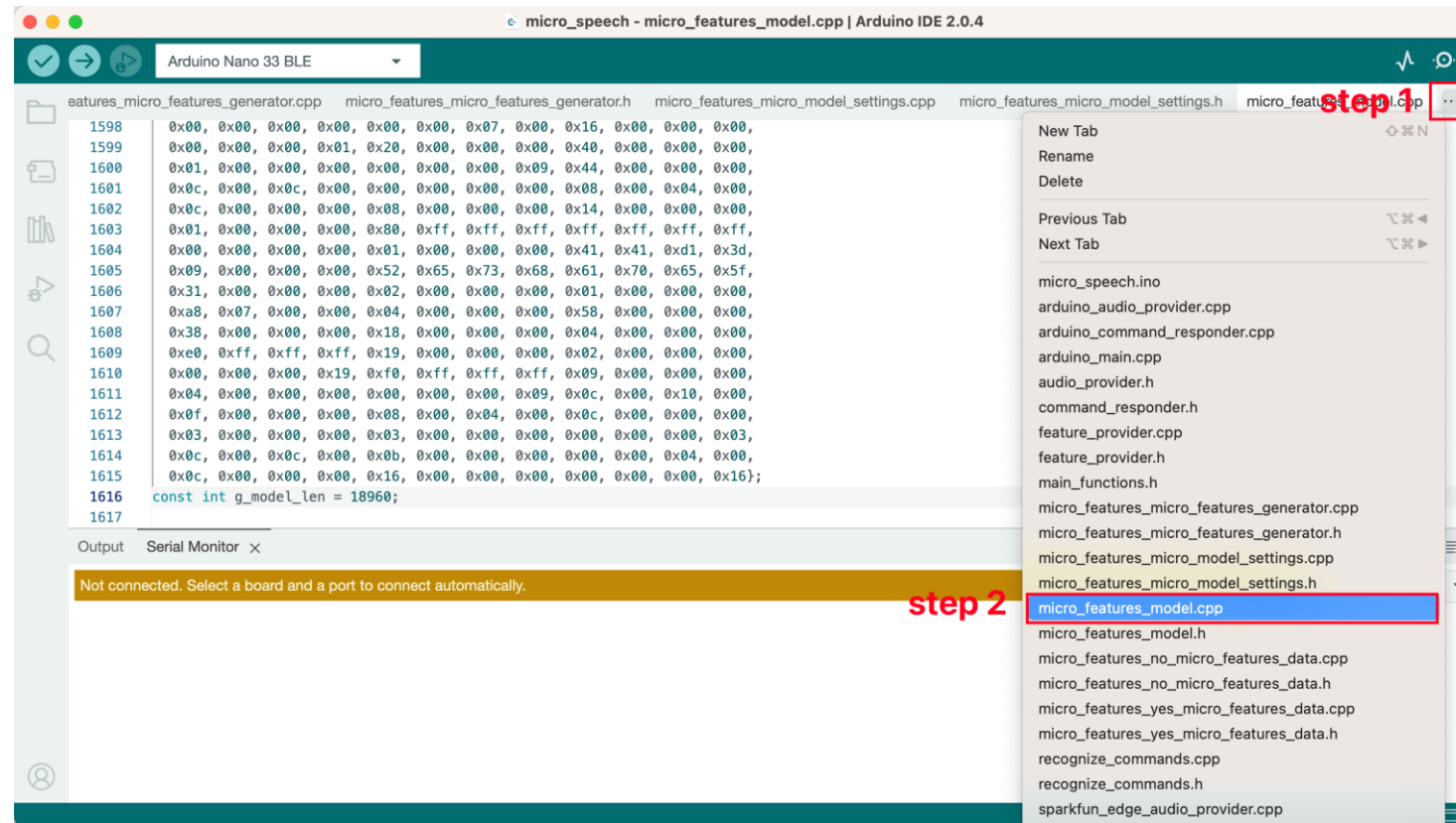
Lab 5 – Hardware

2. From [Arduino IDE](#) open TinyML_EE446 library -> micro_speech example.



Lab 5 – Hardware

3. Click ... on the top bar and open `micro_features_model.cpp`



Lab 5 – Hardware

4. Replace Hex in your `micro_features_model.cpp` with your generated Hex values from the last coding section in *TinyML_Lab5.ipynb*.

Note: Please refer to the generated `kws.cc` file after running the above section. The output shown in the cell only displays a small portion of the last few byte values related to your model.

5. Don't forget to replace `g_model_len` with what you get in the last cell in *TinyML_Lab5.ipynb*

```
0x0c, 0x00, 0x00, 0x00, 0x16, 0x0  
};  
unsigned int g_model_len = 18960;
```

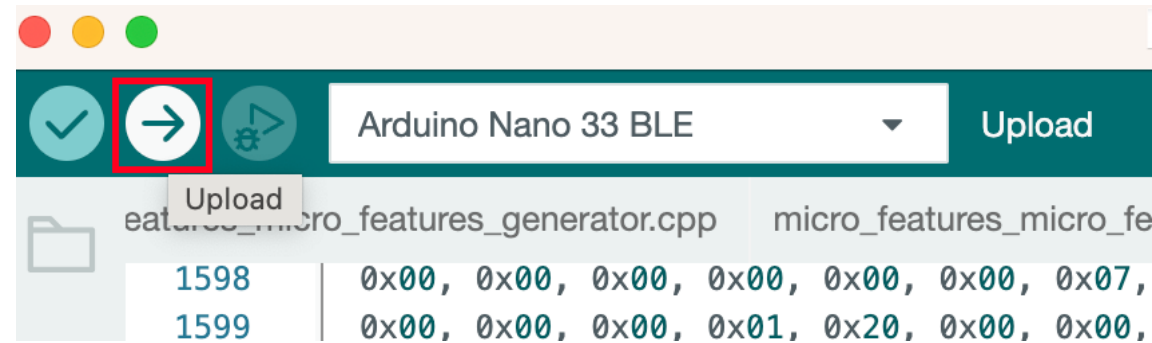


```
0x0c, 0x00, 0x0c, 0x00, 0x0b, 0  
0x0c, 0x00, 0x00, 0x00, 0x16, 0  
const int g_model_len = 18960;
```

Lab 5 – Hardware

5. Save files (**You may save it to another place as examples are read-only**).

And, upload it to Arduino Nano 33 BLE.



After pressing reset, you can now **open serial monitor** to see the KWS result !